

quizgen

Automated Test Assembly from a Question Bank

Selecting blueprint-compliant exams from a pool of
tagged candidate questions via optimisation

Implementation Report

Master's Thesis — KWARC Group
Friedrich-Alexander-Universität Erlangen-Nürnberg

This report documents the implemented logic of the **quizgen** codebase — a selection-only tool that turns an existing question bank into exam papers — the algorithms behind each stage, and the literature each technique draws on.

June 24, 2026

Contents

1	Overview	2
2	Data model and contracts	2
3	The blueprint	3
4	Automated Test Assembly	4
5	Deduplication	5
6	Validation	5
7	Command-line interface	5
8	Provenance of techniques	6
9	Summary	7

1 Overview

quizgen takes an existing **question bank** — a JSON list of questions, each carrying metadata (chapter, difficulty, type, Bloom level) — and *selects* from it the subset(s) that form a professor’s exam. The exam shape is described by a **blueprint**: how many questions in total, how they split across chapters, the target difficulty mix, which question types are allowed, and how many parallel, non-overlapping *versions* to produce.

The scope is deliberately narrow: **there is no question generation and no document ingestion**. The tool reasons only over the metadata of questions it is given and the constraints it is asked to satisfy. This makes selection a well-posed combinatorial optimisation problem with an exact, auditable solution — in contrast to prompting a language model to “write the exam”, which couples creation with constraint satisfaction and gives neither guarantees nor traceability.

Why model selection as optimisation. Hitting a quantitative specification exactly — “10 from Chapter 1, 6 from Chapter 2, 4 from Chapter 3; an 8/8/4 easy/medium/hard split; two versions that share no question” — is a constraint-satisfaction-and-optimisation task. Solving it with a Mixed-Integer Program gives:

- **Exactness** — the blueprint counts are met to the unit, or the run reports precisely which constraint had to be relaxed.
- **Multiple disjoint versions** — produced in a single solve, so parallel exams provably share no items.
- **Quality-aware tie-breaking** — among feasible selections, the one maximising a question-quality objective is chosen.
- **Reproducibility** — the assembled exam embeds the resolved blueprint, so it is self-describing.

The stages. Figure 1 shows the flow. The pool is optionally de-duplicated, assembled into one or more versions against the blueprint, and each version is optionally validated. The stages map onto the modules in Table 1.

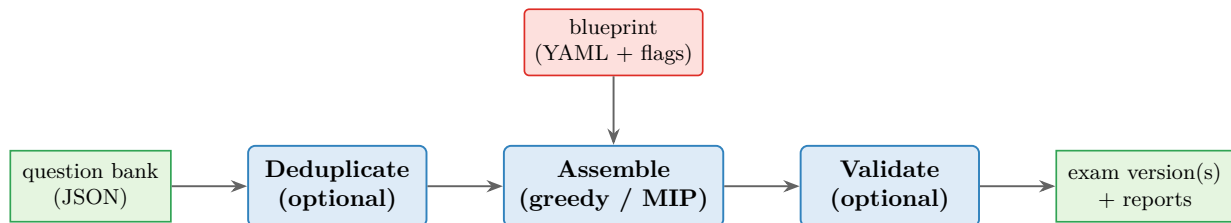


Figure 1: The selection pipeline. Green = data, blue = processing stage, red = professor input. The blueprint drives assembly; dedup and validation are opt-in.

2 Data model and contracts

Every artifact is a Pydantic v2 model (`schema.py`), so the data contract is the single source of truth and the JSON Schema is *exported* from it (committed under `schemas/`) rather than hand-written — the two cannot drift. The central type is `Question`:

```
class Question(BaseModel):
    id: str # uuid4 hex[:12]
    chapter: int # >= 1
    section: str | None
    qtype: QuestionType # mcq | true_false | short_answer | cloze
    bloom_level: BloomLevel # remember .. create (revised Bloom, 2001)
    difficulty: Difficulty # easy | medium | hard
    stem: str # >= 10 chars
```

Table 1: Stage-to-module mapping.

Stage	Module	Responsibility
Data model	schema.py	Pydantic models (Question, Quiz); JSON-Schema export.
Blueprint	blueprint.py	Blueprint model; count resolution (largest-remainder).
Deduplicate	dedup.py, similarity.py	Near-duplicate detection (Jaccard / embedding); pool dedup.
Assemble	assemble.py	Automated Test Assembly: greedy and MIP selectors.
Validate	validate.py, textmatch.py	Schema, blueprint, grounding, duplicate checks.
CLI	__main__.py	Pool + blueprint/flags → exam version(s).
Config	config.py	Single optional embedding-model setting.

```

options: list[str] | None
answer: str           # >= 1 char
explanation: str        # >= 10 chars
source_ref: str        # provenance (free-form)

```

The metadata fields are exactly what selection reasons over (Table 2). The three enumerations make a tag impossible to mistype. A Quiz wraps a list of questions plus the blueprint that produced it, so an assembled exam is self-describing.

Table 2: How each metadata field drives selection.

Field	Used for
chapter	per-chapter quotas
difficulty	the easy / medium / hard mix
qtype	allowed-type filtering
bloom_level	the quality objective (favours discriminating levels)
stem/answer/options	near-duplicate detection
source_ref	provenance / audit trail

3 The blueprint

A *blueprint* (blueprint.py) is the small YAML a professor edits — a *table of specifications* for the exam:

```

title: "AI-1 Midterm Exam"
total_questions: 20
chapters: { 1: "50%", 2: "30%", 3: "20%" }
difficulty_mix: { easy: 0.4, medium: 0.4, hard: 0.2 }
allowed_qtypes: [mcq, true_false, short_answer, cloze]
versions: 2
selector: mip           # greedy | mip
dedup_similarity: 0.85

```

Exact integer resolution. Chapter and difficulty amounts may be integer counts, fractions, or "NN%" strings. Blueprint.resolve() converts them to exact integer counts that sum to total_questions using the **largest-remainder (Hamilton)** method:

$$\text{raw}_k = w_k \cdot N, \quad \text{floor}_k = \lfloor \text{raw}_k \rfloor, \quad r = N - \sum_k \text{floor}_k, \quad (1)$$

after which the r leftover units go to the k with the largest fractional remainders $\text{raw}_k - \text{floor}_k$. This guarantees, for instance, that “40/40/20 of 20” resolves unambiguously to 8/8/4.

YAML and flags (flags win). The same constraints can be supplied or overridden on the command line, so a base `blueprint.yaml` can be tweaked per run. `build_blueprint` loads the YAML (if any) into a dict and overwrites field-by-field with any provided flags before validating the Blueprint:

```
python -m quizgen --pool questions.json --blueprint blueprint.yaml \
    --versions 3 --selector greedy           # override just these two
```

4 Automated Test Assembly

Selection is *Automated Test Assembly* (ATA): choose, from the pool, the subset(s) satisfying all blueprint constraints while maximising overall quality. Two interchangeable selectors sit behind one `Selector` interface (chosen by `blueprint.selector`):

GreedySelector

A fast, content-balancing greedy fill. Honours per-chapter counts, respects the difficulty marginal in a two-pass fill (quota first, then remainder), and skips near-duplicates and questions already used in another version. A good baseline with no optimality guarantee.

MIPSelector

A Mixed-Integer Program (PuLP / CBC) that meets per-chapter *and* difficulty counts exactly, maximises total quality, keeps parallel versions disjoint, and forbids near-duplicate pairs across versions. This is the exact, auditable selection.

MIP formulation. Let $\mathcal{P}$ be the (qtype-filtered) pool, V the set of versions, and $x_{iv} \in \{0, 1\}$ indicate that question i is placed in version v . With per-chapter targets n_c , per-difficulty targets m_d , a quality weight q_i , and the set D of near-duplicate pairs:

$$\max \sum_{v \in V} \sum_{i \in \mathcal{P}} q_i x_{iv} \quad (\text{maximise total quality}) \quad (2)$$

$$\text{s.t.} \quad \sum_{i: \text{chap}(i)=c} x_{iv} = n_c \quad \forall v \in V, \forall c \quad (3)$$

$$\sum_{i: \text{diff}(i)=d} x_{iv} = m_d \quad \forall v \in V, \forall d \quad (4)$$

$$\sum_{v \in V} x_{iv} \leq 1 \quad \forall i \in \mathcal{P} \quad (5)$$

$$\sum_{v \in V} x_{iv} + \sum_{v \in V} x_{jv} \leq 1 \quad \forall (i, j) \in D \quad (6)$$

Constraint (3) fixes the per-chapter blueprint counts; (4) fixes the difficulty mix; (5) makes the parallel versions *disjoint* (no question reused); (6) forbids any near-duplicate *pair* from appearing anywhere across the assembled set. The objective (2) ranks otherwise-feasible selections by a cheap quality proxy (`question_quality`: rewards substantive explanations, well-formed 4-option MCQs, and the more discriminating Bloom levels).

Graceful infeasibility. If the exact program is infeasible for the given pool (e.g. too few hard questions in some chapter), the solver *relaxes* the difficulty equalities (4), records `difficulty_relaxed` in the diagnostics, and re-solves — the per-chapter counts and totals are never silently violated. Capacity warnings are emitted up-front when the pool is too small to honour the demand.

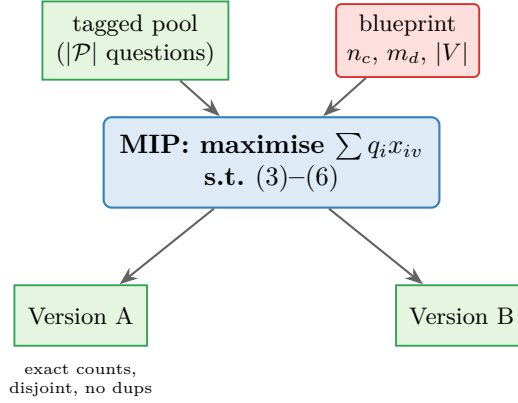


Figure 2: MIP assembly: one optimisation produces all parallel versions at once, guaranteeing they are disjoint and individually blueprint-compliant.

5 Deduplication

Near-duplicate questions waste blueprint slots and make parallel versions predictable. `similarity.py` provides one API (`find_near_duplicate_pairs`) with two backends:

- **Token-Jaccard** over the canonical question text (stem + answer + MCQ options) — dependency-free, always available:

$$\text{sim}(q_1, q_2) = \frac{|T(q_1) \cap T(q_2)|}{|T(q_1) \cup T(q_2)|},$$

where $T(\cdot)$ is the set of lowercased word tokens.

- **Embedding cosine** using an optional sentence-transformers encoder, which catches paraphrases the lexical measure misses. Cosine similarity is computed on L_2 -normalised vectors.

Pairs scoring at/above a threshold are returned as index pairs. The optional pool-dedup pass (`dedup.py`, CLI `-dedup`) drops the later member of each pair *before* assembly; independently, the assembler forbids any surviving near-duplicate pair across versions (constraint (6)). The backend degrades gracefully: with no encoder available it uses the offline token measure automatically.

6 Validation

`validate_exam` (`validate.py`, CLI `-validate`) produces a structured report over four dimensions:

1. **Schema validity** — every question round-trips through the `Question` model.
2. **Blueprint compliance** — per-chapter counts, difficulty mix, total, and allowed qtypes match the blueprint *exactly*.
3. **Grounding coverage** — share of questions carrying a `source_ref` (and, when source text is supplied, whose answer words overlap the referenced passage).
4. **Duplicate rate** — share of questions involved in a near-duplicate pair.

The overall gate `passed` is true only if no schema-invalid items exist and (if checked) the blueprint is satisfied. The grounding check reuses the dependency-free `extract_content_words` helper in `textmatch.py`, kept separate so validation pulls in no heavy dependencies.

7 Command-line interface

`__main__.py` is the single entry point. It loads the pool, builds the blueprint from the YAML and/or flag overrides, optionally de-duplicates, assembles, writes one `Quiz` JSON per version (each embedding the resolved blueprint), and — with `-validate` — writes a Markdown report per version. It exits non-zero if any version is non-compliant, so it composes in scripts and CI.

```
# Pure command line: 20 questions, 10/6/4 across chapters, 40/40/20
# difficulty, two disjoint versions, optimal (MIP) selection:
python -m quizgen --pool questions.json \
    --total 20 --chapters 1:10,2:6,3:4 \
    --difficulty 0.4,0.4,0.2 --versions 2 --selector mip

# Blueprint file + dedup + per-version validation report:
python -m quizgen --pool questions.json --blueprint blueprint.yaml \
    --dedup --validate --out-dir out
```

The assembly and validation steps also expose their own entry points (`python -m quizgen.assemble`, `python -m quizgen.validate`) for use as standalone tools.

8 Provenance of techniques

The table below states, for each implemented technique, the body of literature it draws on. *Honesty note:* the citation keys are the standard references for each method and **should be replaced with the exact bibliography entries the thesis adopts**. The right-hand column distinguishes ideas from the literature from project-specific engineering.

Implemented in code	Technique / idea	Source lineage
<code>assemble.py</code> (MIPSelector)	Automated Test Assembly as 0–1 MIP	van der Linden, <i>Linear Models for Optimal Test Design</i> (2005); exam assembly as integer programming.
<code>question_quality</code> objective	Objective-weighted selection in ATA	van der Linden (2005); here a lightweight quality proxy rather than IRT information.
<code>blueprint.py</code>	Test blueprint / table of specifications	Classical assessment design; content $\times$ cognitive-level specification driving item selection.
BloomLevel metadata	Revised Bloom’s taxonomy	Anderson & Krathwohl (2001), revising Bloom (1956); cognitive-level tag used in the quality objective.
<code>similarity.py</code> (embedding cosine)	Semantic near-duplicate detection	Sentence-BERT embeddings (Reimers & Gurevych, 2019).
<i>Project-specific engineering (not from a paper):</i>		
<code>_largest_remainder</code>	Hamilton / largest-remainder apportionment	Standard apportionment arithmetic; ensures integer counts sum exactly.
<code>GreedySelector</code>	Greedy content-balanced baseline	Engineering baseline to contrast against the MIP.
Jaccard <code>textmatch.py</code>	fallback, Lexical token overlap	Dependency-free fallbacks for offline operation.

Implemented in code	Technique / idea	Source lineage
YAML+flag blueprint, CLI	Configuration / override precedence	Standard tooling; flags override file defaults.

9 Summary

`quizgen` implements the *selection* half of automated exam construction as an exact optimisation problem. Given a tagged question bank and a professor’s blueprint, it assembles blueprint-compliant, mutually disjoint exam versions via a 0–1 Mixed-Integer Program, with a greedy baseline behind the same interface; an optional deduplication pass and a layered validation report (schema, blueprint, grounding, duplicates) verify the result. The tool has no LLM or document-processing dependencies, runs fully offline, and exposes its constraints through a blueprint file and/or command-line flags.